

LoopFuzz: Closed-Loop Runtime Admission Control for LLM-Guided Stateful Protocol Fuzzing

Ketang Chen^a, Xiaolei Ren^{a,*}

^aMacau University of Science and Technology, Avenida Wai Long, Taipa, Macau, China

Abstract

Stateful protocol fuzzing is difficult because a syntactically valid message can still violate the current session state, especially when a large language model (LLM) can inject candidates into a long-running campaign. We present LoopFuzz, a closed-loop runtime admission-control design for LLM-guided stateful protocol fuzzing. LoopFuzz treats model outputs as bounded hypotheses rather than queue entries. It admits an intervention only when runtime evidence supports parseability, response-grounded acceptability, state reachability, and downstream gain. Failed executions become counterexamples for local refinement, and state-aware scheduling triggers model assistance only under adaptive plateau conditions. Re-audited run summaries answer a scoped lineage question: within the AFLNet/ChatAFL implementation and measurement lineage, the integrated LoopFuzz control strategy improves state exploration and improves absolute branch coverage on most primary targets. Auxiliary NSFuzz artifacts and a Mosquito-only MBFuzzer worker archive are retained as external diagnostics rather than homogeneous rankings. The archive does not include same-format StateAFL, Bleem, Logos, or MBFuzzer rows for the primary table, nor a full no-admission ablation. The evidence therefore supports a bounded claim about the evaluated integrated strategy, not an isolated proof of the admission formula or an overall advantage over specialized state-aware fuzzers.

Keywords: Protocol fuzzing, Large language models, Runtime admission control, Stateful testing, Counterexample-guided refinement

1. Introduction

Network protocol implementations remain difficult security targets because correctness depends on interaction history rather than isolated inputs [1–5]. An FTP server, for example, rejects a well-formed `PASS` command if no valid `USER` command has established the login state. Unlike file fuzzing, protocol fuzzing must preserve message syntax, temporal order, and request-response progress across a multi-step session [5–10].

Traditional coverage-guided fuzzers work well for stateless parsers. They fit network servers poorly when later behavior depends on earlier valid exchanges [11–15]. Stateful greybox fuzzers such as AFLNet and StateAFL partially close this gap. They infer progress from responses and replay valid traces [1–4, 7, 16, 17]. Even so, byte-level mutation still struggles to synthesize the context-sensitive messages needed to cross deep protocol boundaries.

Large language models offer a plausible source of such messages. ChatAFL and later LLM-assisted fuzzers show that a model can extract structure from specifications, logs, or documentation and produce fluent candidates [18–25].

But fluency is not enough. A request can be grammatically plausible and still be invalid for the live session state. Such executions spend budget without improving queue quality.

LoopFuzz places a runtime admission controller between LLM generation and queue admission. It treats the LLM as a hypothesis generator rather than an oracle. It derives message hypotheses from available RFC fragments, seed traces, and observed responses. It stores failed executions as counterexamples and uses them for counterexample-guided refinement (CEGR) rather than unconstrained regeneration. It also uses inferred protocol state machine (IPSM) feedback to prioritize frontier states and rare transitions, and to trigger assistance only when ordinary mutation stalls. The name *LoopFuzz* emphasizes closed-loop admission and refinement, avoiding any suggestion of formal verification of protocol semantics, model behavior, or server correctness.

The contributions of this paper are as follows:

- **Hypothesis-Driven Modeling.** LoopFuzz derives message grammars and field constraints from RFC fragments, seed traces, and observed responses, and maintains them as explicit runtime hypotheses.
- **Runtime Admission Boundary.** LoopFuzz inserts a runtime admission controller between the LLM and the queue. It checks structure before execution and uses bounded trial evidence to evaluate

*Corresponding author: Xiaolei Ren. Email address: xlren@must.edu.mo

Email addresses: 3250006913@student.must.edu.mo (Ketang Chen), xlren@must.edu.mo (Xiaolei Ren)

response acceptability, state reachability, and downstream gain before durable queue promotion. The current archive evaluates this boundary as part of an integrated controller rather than through a full no-admission ablation.

- **Counterexample-Guided Refinement.** LoopFuzz preserves failed request-response traces as counterexamples and uses them to patch one field constraint or production at a time, which constrains model freedom and stabilizes refinement.
- **State-Aware Scheduling.** LoopFuzz leverages IPSM topology, state productivity, and adaptive plateau detection to prioritize underexplored frontier states and to request model assistance only when mutation alone stalls.
- **Scoped Empirical Study.** We evaluate LoopFuzz on a repeated-run benchmark archive. The primary study asks whether the integrated control strategy changes final `b_abs` and response-derived IPSM edges within the AFLNet/ChatAFL lineage. Auxiliary NSFuzz artifacts, Mosquitto-only MBFuzzer artifacts, and policy ablations are reported as scope-setting evidence rather than as a claim of overall superiority over StateAFL, NSFuzz, Bleem, Logos, MBFuzzer, or other specialized state-aware fuzzers. Because the archive lacks a *LoopFuzz-no-admission* condition, the ablation evidence does not isolate the complete contribution of the *P/U/R/G* admission formula.

Accordingly, the paper evaluates LoopFuzz as a campaign-control policy, not as a standalone message generator. In stateful protocol fuzzing, generation matters only when it survives runtime evidence and improves downstream queue quality.

2. Background and Motivation

Table 1 contrasts LoopFuzz with representative fuzzers. Prior LLM-assisted protocol fuzzers improve input fluency, but they leave admission largely open-loop. We study a narrower question: does an integrated control strategy centered on runtime validation make LLM proposals more useful in a long-running stateful campaign?

2.1. Coverage-Guided Fuzzing vs. Network Targets

A coverage-guided fuzzer operates through a repeated feedback cycle of seed selection, mutation, execution, coverage measurement, and novelty-based queue update [12–15]. This loop is effective for file parsers, but network targets impose three constraints that matter here:

1. **Temporal state dependence:** server behavior depends on the order and semantics of earlier messages, not only on the current payload [1–4].

2. **Strict message format:** protocols require delimiters, field layouts, and often checksum or length consistency, so many mutated messages die before they reach deeper logic [4, 26, 27].
3. **Execution instability:** sockets, forking daemons, and asynchronous response handling complicate both coverage collection and campaign throughput [17].

2.2. Stateful Greybox Fuzzing (SGF)

SGF addresses this gap by treating the target as an *inferred protocol state machine* (IPSM). Formally, the IPSM can be viewed as a Mealy machine $M = (Q, I, O, \delta, \lambda, q_0)$, where:

- Q is the set of inferred protocol states;
- I is the set of client messages;
- O is the set of observed server responses;
- δ maps a state and input to a successor state;
- λ maps a state and input to an observed response; and
- q_0 is the initial connection state.

AFLNet-style fuzzers infer Q from observed responses and use that structure to bias later exploration toward rare or promising parts of the protocol. Here, the IPSM matters for one reason. It gives LoopFuzz a notion of progress that is richer than raw edge coverage alone.

2.3. LLM Integration into Protocol Fuzzing

Large language models make it easier to propose structure-aware protocol messages from RFC text, interaction history, or observed responses. That helps with syntax and message diversity, but it does not solve the harder control problem: a fluent request can still be wrong for the live session state.

2.4. A Motivating Example: Semantic Drift

Consider an FTP fuzzing campaign against ProFTPD, as illustrated in Fig. 1. The fuzzer executes `USER` and `PASS`. It then queries the LLM for the next request. The model returns `RETR example_file.txt`. The request is syntactically fluent but semantically premature. The client lacks an active data channel via `PASV` or `PORT`. The server rejects it with `425 Use PORT or PASV first`.

We call this failure *semantic drift*. The request is locally plausible, but it violates the precondition of the current session state. An open-loop LLM fuzzer admits the request because it looks fluent. It then mutates a rejected branch instead of crossing the missing state transition. LoopFuzz handles this drift through runtime validation rather than through a purely pre-execution oracle. The controller first checks local structure and parseability, then performs a bounded trial execution whose `425` reply becomes negative evidence. The candidate is not promoted into the long-lived

Table 1: Among the listed systems, LoopFuzz combines LLM hypotheses with runtime admission and online refinement; representative prior fuzzers differ in input prior, state signal, and admission boundary.

System	Input prior	State signal	Admission rule	Online refinement
Coverage-guided mutation fuzzing [11, 12]	Byte mutation	None	None	No
AFLNet [1]	PCAP mutation	Response	Novelty	No
StateAFL [2]	PCAP mutation	Memory	Novelty	No
ChatAFL [18]	LLM requests	History text	Direct admission	No
LoopFuzz (ours)	LLM hypotheses	IPSM frontier + validation	Runtime admission	Yes

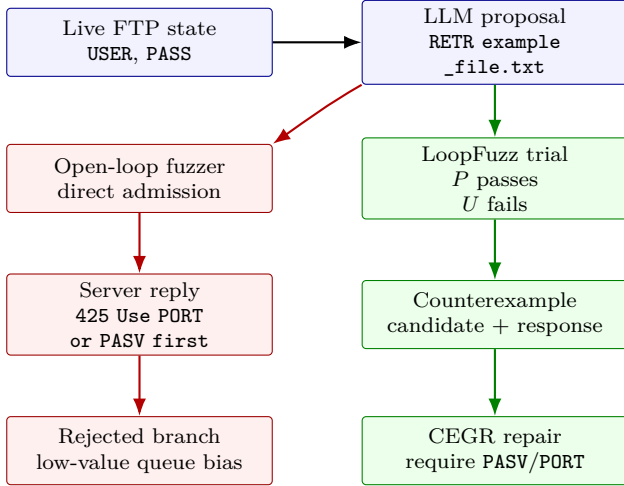


Figure 1: Semantic drift in an FTP session. The model proposes RETR before the session opens a data channel. Open-loop fuzzing admits the rejected branch. LoopFuzz first checks local structure, then uses a bounded trial execution to observe the 425 response. The candidate is not promoted into the queue; the trace becomes a counterexample for repairing the missing PASV/PORT precondition.

queue, and the failed trace is recorded as a counterexample. CEGR then repairs the missing PASV/PORT precondition instead of regenerating the full continuation. This example motivates the design claim: LLM assistance should be mediated by runtime evidence rather than by surface fluency alone. We use semantic drift as an operational motivation for selective queue admission and CEGR repair, not as a separately estimated global rate.

3. Related Work

We position LoopFuzz against the prior lines that are closest in mechanism and evaluation.

Earlier structure-aware fuzzing used explicit structure, inferred snippets, token constraints, or grammar guidance to preserve validity [26–32]. These systems improve input quality, but they do not address the control problem that arises when an external model proposes state-dependent protocol messages.

Stateful greybox and state-aware network-service fuzzing address protocol interaction more directly. AFLNet, StateAFL, NSFuzz, and the broader SGF line infer or expose state structure from server responses, process

memory, or program variables [1–3, 16, 33]. They then use that structure to prioritize rare or productive traces. These systems define the broader comparison landscape, but they do not all expose the same state abstraction, instrumentation path, or archived endpoint metrics.

Later systems extend this idea to packet-sequence exploration, format-sensitive protocol fuzzing, and communication-heavy environments [4–8, 34–38]. LoopFuzz asks a narrower question: given an AFLNet-style response-derived IPSM and the ChatAFL-style LLM proposal path, does an integrated runtime-admission strategy make the campaign more reliable? AFLNet and ChatAFL are therefore the closest implementation-lineage baselines. StateAFL, SGF, NSFuzz, Bleem, Logos, and MBFuzzer remain important external comparison points, but the archive lacks same-format reproductions for most of them, gives NSFuzz partly different artifact semantics, and contains MBFuzzer only as a Mosquito worker archive without AFLNet-lineage edges. We treat that gap as a limitation and use the external rows to bound the claim.

Adjacent work studies state-aware or interaction-heavy fuzzing in robotic policies, firmware, Linux drivers, middleware, hardware, and embedded systems [39–45]. Although not all are protocol fuzzers, they reinforce the point that state feedback can dominate raw mutation quality.

Target-aware scheduling and semantic validation form another relevant line. Directed greybox fuzzing, seed selection, and strategy adaptation study how to steer budget toward high-value behaviors [11–15, 46, 47]. Semantic validation and fuzz-driver generation show that stronger inputs often require an explicit acceptance interface [48, 49]. LoopFuzz applies that lesson to LLM proposals.

Work on dependency inference, data-flow guidance, and learned signals outside protocol fuzzing also supports treating guidance as bounded evidence rather than as an oracle [50–52].

LLM guidance addresses the synthesis bottleneck more directly. ChatAFL uses RFCs and interaction histories to generate protocol messages [18]; adjacent systems use LLMs for general input generation, documentation-guided constraints, device fuzzing, or fuzz-driver generation [19–23, 53]. Our work is closest to ChatAFL, but adds a control boundary: LoopFuzz asks whether a proposal should be admitted, whether its hypothesis survives runtime evidence, and how the IPSM should guide the next query.

Benchmarking practice is also part of the context. Pro-

FuzzBench established a reusable baseline for repeated protocol-fuzzing experiments [16]. Recent methodology work shows how easily comparisons become unstable without fixed budgets, repeated trials, and careful interpretation of coverage signals [54–56]. We follow that guidance because runtime-validated assistance is especially sensitive to campaign history.

4. LoopFuzz Design

Fig. 2 summarizes the three coupled loops in LoopFuzz: the Grammar Hypothesis Loop, the Fuzz Execution Loop, and the LLM Intervention Loop.

LoopFuzz preserves the stateful greybox fuzzing execution path. It sets a strict control boundary between LLM generation and durable queue admission. A model output changes the long-lived campaign only after runtime evidence supports admission. The admission rule has two stages: structural and local parseability checks run before a candidate is tried, and response behavior, inferred state reachability, and downstream campaign value are evaluated after a bounded trial execution. The method has four coupled components: grammar hypothesis generation, runtime admission control, counterexample-guided refinement, and state-aware scheduling.

This control boundary is the core idea of the paper. ChatAFL is open-loop: it uses a fixed trigger, a plaintext prompt, and direct admission of the extracted request. LoopFuzz is closed-loop: IPSM growth, state productivity, and hypothesis fitness determine when the LLM is queried, what context it receives, and whether a tried proposal is promoted into the campaign.

The three loops in Fig. 2 define the analytical roles of the method. The Grammar Hypothesis Loop constructs and repairs message hypotheses from RFC knowledge, seed traces, and observed responses. The Fuzz Execution Loop performs state selection, mutation, bounded execution, IPSM update, and novelty checking. The LLM Intervention Loop is activated only under plateau conditions and returns either a validated continuation or a bounded control action.

The remaining labels in Fig. 2 denote control interfaces rather than independent claims. Tier-1 sampling checks parsed execution regions, Tier-2 refinement repairs counterexamples, schema validation checks model replies, and bounded action dispatch maps an admitted action to mutation, seed-priority, or target-state control. The prompt reuse filter and query isolation bound repeated or failed model calls.

4.1. Grammar Hypothesis Loop

Let $\mathcal{L}(c, s, \Sigma)$ denote the LLM generation function under prompt context c , state summary s , and protocol knowledge Σ . LoopFuzz uses $\mathcal{L}$ in two stages. During initialization, it preserves ChatAFL’s template extraction and seed enrichment path. Initial enrichment is scheduled as a bounded phase so that corpus diversification does not dominate early

campaign time. During fuzzing, LoopFuzz adds a lazily initialized hypothesis layer. This layer is activated at the first plateau event and updated through sampled validation and refinement. The separation is important. The initialization phase broadens the initial seed set. Runtime hypotheses support runtime-validated control during long campaigns.

The Grammar Hypothesis Loop draws on three evidence sources when they are available: RFC fragments, queued seed traces or packet examples, and observed server responses, including response codes and error texts. For each message type m , the loop maintains a hypothesis $h_m = (G_m, \Phi_m)$. Here, G_m is an ABNF-style or template-style production, and Φ_m is a set of field constraints. In LoopFuzz, Φ_m captures length bounds, enumerated values, token dependencies, and simple cross-field relations. This representation is more constrained than free-form text and lighter than a full formal protocol specification. We choose it because sampled online validation must remain cheap.

The evidence sources need not be complete. RFC fragments supply structure, seed traces provide executable examples, and observed responses provide target-specific evidence about rejection, progress, and state change. LoopFuzz therefore requires testable hypotheses, not a full formal protocol specification; runtime admission decides whether those hypotheses can affect the campaign.

4.2. Runtime Admission Controller

The runtime admission controller is the central mechanism in our design. It does not formally prove correctness, and it is not a purely pre-execution filter. Instead, it makes queue-admission decisions through two validation stages. Stage 1 checks whether a model proposal is structurally well formed and locally parseable before it is sent. Stage 2 performs a bounded trial execution and then evaluates the observed response, inferred state movement, and coverage or IPSM gain. A candidate intervention x is retained only if

$$A(x) = P(x) \wedge U(x) \wedge R(x) \wedge G(x),$$

where P denotes pre-trial local parseability, while U , R , and G are post-trial predicates computed from the bounded execution: U denotes response-grounded acceptability, R denotes evidence of new or redirected state reachability, and G denotes downstream gain in IPSM structure or coverage. If P fails, the proposal is not executed. If P passes but any post-trial predicate fails, the trial trace can be recorded as a counterexample or used to update state productivity, but the candidate is not promoted as a durable queue entry or accepted control action.

Table 2 gives the operational definition used to make this rule reproducible. The table separates the pre-trial structural gate from the post-trial evidence gates and lists the minimum event fields needed to replay an admission decision. These are the event-level fields that a complete artifact should preserve; as discussed in Section 8, the current archive does not contain normalized admission-event

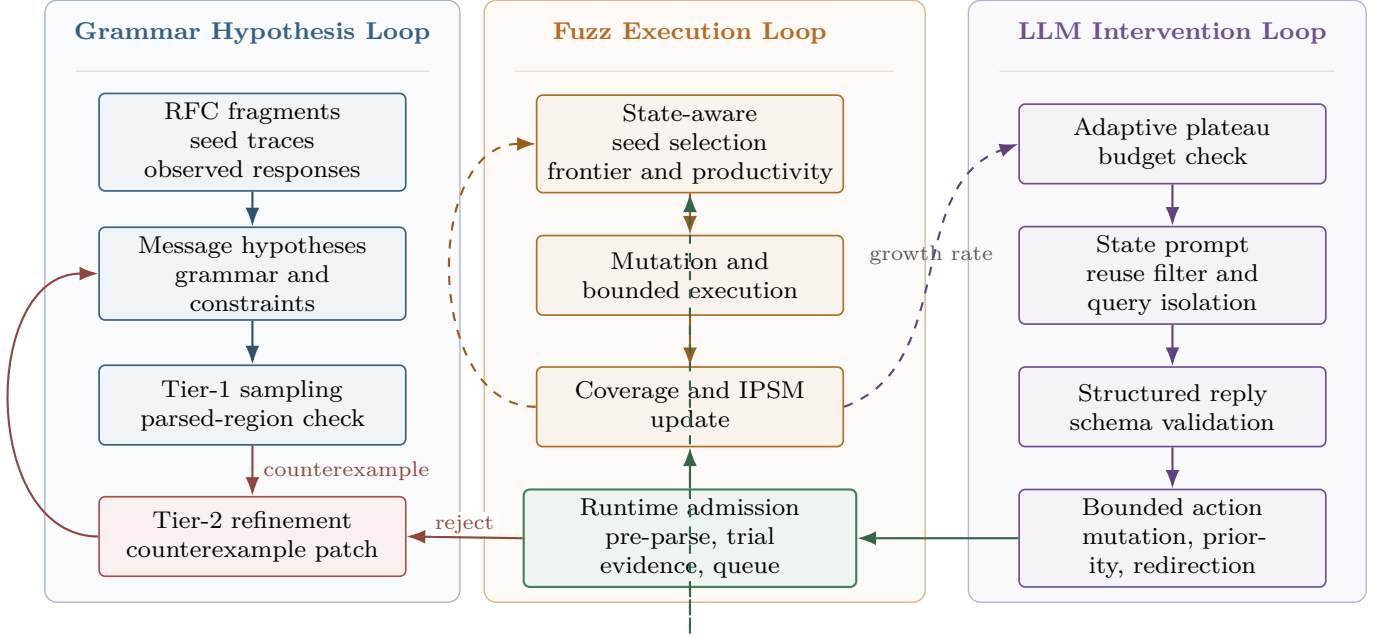


Figure 2: LoopFuzz changes the LLM from a queue producer into a bounded policy proposer. The Grammar Hypothesis Loop maintains message hypotheses, the Fuzz Execution Loop updates coverage and IPSM feedback, and the LLM Intervention Loop proposes bounded interventions. Runtime admission is two-stage: structural checks run before trial execution, while response, state, and coverage checks decide afterward whether the proposal can affect the queue.

logs from which cross-target admission-rate summaries can be reconstructed.

The distinction between R and G is intentional. R asks whether the candidate can reach a meaningful protocol state under the response-derived IPSM. G asks whether that execution produced measurable campaign novelty. A trial can therefore pass U and R yet fail G when it reaches an already-known state without new coverage, IPSM, or queue value.

The cleanest ablation for this design would be *LoopFuzz-no-admission*: it would keep the same model endpoint, prompts, query budget, plateau triggers, and scheduling policy, but skip the $P/U/R/G$ checks and directly enqueue or apply the model’s suggested request or control action. Such a condition would isolate the contribution of the admission boundary from the rest of the control strategy. The current archive does not contain that condition. Therefore, our experiments evaluate LoopFuzz as an integrated control policy and do not independently prove the full causal effect of the admission formula.

The FTP example in Fig. 1 illustrates the same two-stage logic. The candidate `RETR example_file.txt` can satisfy local syntax under P , so LoopFuzz may try it in a bounded validation execution. The live 425 response then makes U fail because no data channel has been negotiated. The trial also provides no useful new transition or coverage signal, so R and G do not support admission. LoopFuzz therefore declines to promote the candidate into the queue and records a counterexample that constrains CEGR to repair the missing `PASV/PORT` precondition.

4.2.1. Structured Output and Parseability

The plateau stage accepts either a concrete next-request candidate or a bounded structured-action object. Runtime validation first checks structural well-formedness, field types, length bounds, delimiter or terminator consistency, and action-schema consistency. LoopFuzz then tests local parseability against the current message hypotheses. It reuses parsed message regions already available during ordinary execution. This first stage can reject structurally invalid interventions before any trial execution. It does not decide whether a syntactically valid request is useful; that decision depends on the post-trial predicates in Table 2.

4.2.2. Response-Grounded Acceptability

Parseability alone is insufficient. A parsed request can still be semantically premature. LoopFuzz therefore classifies the response observed during the bounded trial execution and treats a non-rejection response as positive evidence. For response-code protocols, 4xx/5xx states are the default rejection class; timeouts, early disconnects, unparsable replies, and explicit error texts are also negative evidence. The classification is conservative rather than absolute: a target adapter or accumulated productivity evidence can keep an apparent error state from becoming a permanent dead end if it repeatedly leads to new paths. Rejected or persistently unproductive responses reduce state productivity and hypothesis fitness. This gives an execution-grounded notion of acceptability rather than a semantic proof. The controller asks a narrow post-trial question: did live target behavior support promoting this candidate into the campaign?

Table 2: Operational definition of the $P/U/R/G$ admission predicates. P is checked before a trial execution. U , R , and G are computed from the bounded trial response, inferred state sequence, and coverage or IPSM novelty. A failed post-trial predicate can update counters, state productivity, or CEGR counterexamples, but it does not create a durable queue entry.

Pred.	Input	Decision rule	Pass / fail example	Minimum audit fields	Execution and queue effect
P					
Parse.	Schema-validated model reply; cleaned request or bounded action; current hypothesis $h_m = (G_m, \Phi_m)$; parsed message regions.	Pass if the reply has a supported action type, required fields, valid field types and bounds, protocol delimiters or terminators, and a local parse under the current message hypothesis. Fail on malformed JSON/action objects, unknown action types, missing terminators, length/type violations, or local parser failure.	Pass: <code>RETR file.txt</code> matches an FTP request template. Fail: missing command terminator, unknown action, or field length outside Φ_m .	event id; action and message type; schema flag; parse flag; parse error; hypothesis id.	Exec.: no. Failure stops the candidate before execution. Success only makes it eligible for a bounded trial.
U					
Accept.	Trial response bytes, extracted response code or state id, timeout/disconnect status, and protocol adapter.	Pass if the live response is not classified as a rejection sink. For text protocols whose states are three-digit response codes, 4xx/5xx are the default rejection class; target adapters or accumulated productivity evidence can override this when an apparent error state repeatedly yields paths. Timeouts, connection resets before a response, unparseable replies, and explicit <code>ERROR/BAD/invalid/denied</code> texts are negative evidence.	Pass: FTP 230 login success or 227 passive-mode progress. Fail: FTP 425 Use PORT or PASV first, timeout, or reset without response.	response bytes; response code or state id; response class; timeout/disconnect; error hint; productivity; rejection reason.	Exec.: yes, bounded. Failure blocks promotion and may lower hypothesis fitness or create a CEGR counterexample.
R					
Reach.	Trial state sequence extracted from responses, source state, scheduled target state or frontier state, and IPSM before the trial.	Pass if the trial reaches a non-rejection protocol state, reaches the scheduled target state, moves out of a repeated rejection sink, or realizes a new response-derived state relation. Fail if it stays in the same rejection/error sink or produces no usable state sequence. This predicate answers whether the candidate reached a meaningful protocol state, regardless of whether it produced new code coverage.	Pass: a candidate reaches the target passive-mode/data-channel state. Fail: repeated 425 leaves the execution in the same premature-transfer sink.	source state; destination state; state sequence; target state; reached-target flag; new state relation; state productivity.	Exec.: yes, bounded. Positive evidence updates IPSM/frontier statistics. Durable admission still requires G .
G					
Gain	Coverage bitmap result, branch/edge delta, IPSM path or edge novelty, queue-favored status, and the pre-trial campaign state.	Pass if the trial yields measurable campaign novelty: a new coverage bitmap bit or branch, a new IPSM node/path/edge, or a seed that the fuzzer marks as favored because it improves the current corpus. Fail if the candidate is acceptable and reachable but only repeats an already saturated state with no coverage or IPSM novelty.	Pass: <code>has_new_cov</code> or a new IPSM path is observed. Fail: an acceptable NOOP repeats a known state and exposes no new bitmap or IPSM signal.	new bits; coverage flag; branch delta; new IPSM node/path/edge; favored flag; queue-promotion flag; gain reason.	Exec.: yes, bounded. Only candidates passing P , U , R , and G become durable queue entries or accepted controls.

4.2.3. State Reachability and Coverage Gain

The final admission question is whether the trial helped the campaign. LoopFuzz separates the answer into a state predicate and a novelty predicate. Reachability R is computed from the response-derived state sequence: did the candidate reach a non-rejection state, the scheduled target state, or a new state relation? Gain G is stricter and queue-facing: did that execution produce new coverage, a new IPSM node/path/edge, or a favored corpus entry? This separation handles the common case in which a candidate is state-valid but already saturated. The controller therefore uses IPSM-derived reachability and AFLNet-style novelty signals, not text quality alone. Fluent but repeatedly unproductive candidates may still consume a bounded validation execution, but they are filtered out before durable queue promotion instead of accumulating as low-value queue bias.

4.3. Counterexample-Guided Refinement

When sampled validation fails repeatedly, LoopFuzz records a concrete counterexample. The record contains the failure-triggering request fragment, the observed response, and the mismatch against the current hypothesis. Refinement is deliberately local. Instead of rewriting the

full protocol description, the refinement interface constrains each repair to one field constraint or one production fragment. This single-patch discipline narrows the model’s search space, limits unconstrained edits, and makes the repair path auditable. The resulting update is reintegrated into protocol matching and re-evaluated by later sampled validation.

This repair boundary is intentionally weaker than formal grammar repair. The one-field or one-production constraint is enforced through the refinement prompt and audit log, not through hard AST differencing. The current implementation also stores concrete failed traces without counterexample minimization. We therefore claim bounded local refinement, not complete protocol inference.

4.4. State-Aware Scheduling and Plateau-Triggered Assistance

Reducing malformed generations solves only half of the problem. Stateful fuzzers also stagnate when the campaign keeps revisiting locally dense IPSM regions while failing to cross narrow protocol bridges. LoopFuzz therefore treats IPSM feedback as a first-class scheduling signal.

To address this, LoopFuzz augments state selection with a frontier bonus β derived from the out-degree of the IPSM:

$$\beta = \begin{cases} 4.0 & \text{if } \deg^+(s) \leq 1 \\ 2.0 & \text{if } \deg^+(s) \leq 3 \\ 1.0 & \text{otherwise} \end{cases}$$

States absent from the IPSM receive the maximum bonus $\beta=4$. LoopFuzz then modulates this bonus with a three-level error-productivity hint: unknown, likely error, or confirmed productive. The reported implementation encodes these levels as 0, 1, and 2. This hint downweights frontier states that repeatedly behave like rejection sinks, but it does not discard them outright.

In the reported policy, hint level 1 applies a $\beta \times 0.5$ penalty. Hint level 0 is reduced to $\beta \times 0.7$ only after repeated low-productivity selections (*selected_times* ≥ 30 and *productivity* < 0.005). Hint level 2 receives no penalty. The hint is initialized from response-code classification and updated online. As a result, states can be promoted or demoted as evidence accumulates. They are not fixed by an early response pattern. This policy approximates a rare-transition preference without requiring an exact model of the full protocol state space.

These constants are the reported control policy, not a claim of global optimality. The frontier bonuses separate unseen or low-out-degree states from denser regions by coarse preference. The error-productivity penalties and the (30, 0.005) gate reduce repeated investment in response-derived rejection sinks while allowing recovery if later executions become productive. The (80, 30%) node-stagnation safeguard and 64-call intervention budget prevent persistent hot-spot lock-in and unbounded LLM overhead. We report them for auditability, not as a full sensitivity analysis.

A complementary graph stagnation check tracks whether IPSM growth has collapsed at the graph level rather than only at the local state level. When no new nodes appear for an extended interval, the scheduler intermittently injects random selection to avoid persistent hot-spot lock-in. In the reported configuration, that safeguard activates after 80 consecutive rounds without node growth and uses a 30% random-selection probability.

Plateau detection uses the IPSM edge growth rate $\dot{E}$, measured as edges gained per minute. LoopFuzz recalculates this rate every 60 seconds of wall-clock time and uses it to set an adaptive threshold:

$$\tau(\dot{E}) = \begin{cases} 300 & \text{if } \dot{E} > 5 \\ 200 & \text{if } 1 < \dot{E} \leq 5 \\ 150 & \text{if } \dot{E} \leq 1 \end{cases}$$

The initial value is $\tau=200$ for textual protocols. MQTT uses the higher thresholds {500, 400, 300} under the same edge-growth bins because binary framing and broker-mediated behavior expose state progress more sparsely. More generally, LoopFuzz treats plateau triggering as a

rate-sensitive control decision, not as a fixed-frequency query rule.

Assistance is invoked only when edge growth remains weak and the model-intervention budget is not exhausted. In the reported configuration, the plateau path opens when the unproductive streak reaches τ and prior model interventions remain below 64. Once inside that path, hypotheses are initialized lazily on first use. Refinement remains gated by validation count and accumulated counterexamples. The prompt reuse filter covers the most recent 64 LLM invocations. This prevents assistance from reacting to the same state summary again and again.

The plateau prompt exposes a compact state summary rather than a raw execution log. It includes IPSM node and edge counts, the current edge-growth rate, pending favored seeds, queued paths, the current target state, and the least productive states. The admissible action space is also bounded. A model can redirect the target state, reprioritize a small set of seeds, or propose bounded mutations over an existing seed. These actions are not admitted because they are fluent. They must still pass the same runtime admission boundary. This design keeps the LLM in the role of a state-directed policy proposer.

MQTT-specific scheduling is treated as a protocol adaptor, not as the general mechanism of LoopFuzz. For MQTT campaigns, packet-type selection uses a state-indexed action policy, and mutation-family selection uses a bandit-style controller. These signals help expose broker forwarding behavior to the IPSM, but they are protocol-adaptor details rather than primary performance-evaluation evidence. In short, assistance is conditional, budgeted, state-directed, and scoped by protocol evidence.

LoopFuzz also supports protocol-adaptor signals that are not headline evidence. For MQTT, the adaptor translates between binary packets and textual hypotheses, drives broker interactions, and records differential behavior across roles. A lightweight oracle can log authentication bypass, state-order violations, information leaks, traversal patterns, injection patterns, or denial-of-service indicators. These signals support auditing and triage, but they are not treated as confirmed vulnerabilities because crash deduplication, root-cause analysis, exploitability review, and oracle calibration are not normalized.

Algorithm 1 specifies the plateau path and the hypothesis-refinement gate. It formalizes how state-aware triggering, runtime admission checks, and CEGR interact within one campaign loop.

This integration imposes a bounded operating model. Initial enrichment is separated from long-running admission control, and hypothesis construction is deferred until the first plateau event. Model queries use process isolation, and network cleanup is bounded to limit persistent stalls on forking daemons. These mechanisms are operating conditions rather than separate novelty claims.

Algorithm 1: Runtime-Validated Plateau Recovery in LoopFuzz

Input: Target T , LLM interface $\mathcal{L}$, corpus C , edge-growth signal $\dot{E}$
Output: Coverage bitmap $\mathcal{B}_{global}$ and IPSM $\mathcal{M}$

```
1 Initialize  $\mathcal{B}_{global}$  and  $\mathcal{M}$ 
2 while the campaign budget remains do
3    $s \leftarrow \text{SELECTSEED}(C, \mathcal{M})$ 
4    $(trace, Q) \leftarrow \text{MUTATEEXECUTE}(T, s)$ 
5   Update  $\mathcal{B}_{global}$ ,  $\mathcal{M}$ , and frontier scores from  $(trace, Q)$ 
6   if sampled validation is due then
7     Validate active hypotheses on parsed regions of  $Q$ 
8   end
9   if the adaptive plateau gate opens then
10    Initialize hypotheses on first use, if needed
11    if counterexample evidence opens the refinement gate then
12      Patch the lowest-fitness hypothesis locally
13    end
14     $c \leftarrow \text{BUILDPROMPT}(\mathcal{M}, C, Q, \dot{E})$ 
15    if  $c$  is not in the recent prompt cache then
16       $r \leftarrow \mathcal{L}(c)$  under process isolation
17      Admit  $r$  only if parseability, acceptability, reachability, and gain all hold
18      Apply the admitted bounded action to  $C$ 
19    end
20  end
21 end
```

5. Experimental Evaluation Design

We organize the evaluation around three research questions that match the paper’s main claims. The main experiment is a lineage experiment: it asks whether the integrated LoopFuzz control strategy changes a campaign that already has response-derived IPSM feedback and ChatAFL-style LLM proposals. We then use the auxiliary NSFuzz artifact counts and ablation limitations to show where that claim stops.

- **RQ1 (Whole-Campaign Effect):** Does the integrated LoopFuzz strategy avoid the coverage regressions that can arise from direct open-loop generation?
- **RQ2 (IPSM Escape Dynamics):** Does the evaluated integrated policy, with frontier scheduling and plateau-triggered intervention, sustain IPSM growth after the AFLNet/ChatAFL baselines stall?
- **RQ3 (End-of-Campaign Outcome):** Under equal 24-hour budgets in the AFLNet/ChatAFL lineage, how does LoopFuzz affect final **b_abs** and final response-derived IPSM **edges**?

5.1. Experimental Setup

We ran all experiments on a dedicated Ubuntu 20.04 server with 32 CPU cores and 64GB RAM. The benchmark suite spanned 10 implementations across seven protocol families:

1. **FTP:** LightFTP, bftpd, ProFTPD, and Pure-FTPd.
2. **SMTP:** Exim.

3. **RTSP:** Live555.
4. **SIP:** Kamailio.
5. **DAAP/DACP:** Forked-daapd.
6. **HTTP:** Lighttpd1.
7. **MQTT:** Mosquitto.

The benchmark suite contains 10 targets, but the provided run-summary archive supports different comparisons on different subsets. The primary comparison uses AFLNet, ChatAFL, and LoopFuzz on the nine off-the-shelf non-MQTT targets: LightFTP, bftpd, ProFTPD, Pure-FTPd, Exim, Live555, Kamailio, Forked-daapd, and Lighttpd1. This primary comparison is selected because AFLNet supplies the response-derived IPSM lineage and ChatAFL supplies the direct LLM-assisted predecessor that LoopFuzz modifies. The audit is based on the updated primary, ablation, and NSFuzz result archives.

The equal-budget primary subjects are textual request/response protocols. We restrict our performance claims to this regime. The method has not been validated on opaque binary or encrypted protocols such as DNS, TLS, SSH, DTLS, or DICOM.

Both ChatAFL and LoopFuzz used the same pinned GPT-4o-mini endpoint. Prompts remained protocol-agnostic beyond protocol name, observed state, and runtime summary. We did not hand-specialize prompt text per target. The primary table uses approximately 24-hour archived repeated runs under the deterministic selection rule described below, with no per-system cherry-picking of directories or best runs.

Resource isolation and concurrency. Each long-running sample is launched as one Docker container through the ProfuzzBench wrapper. The launcher forwards the container count, timeout, skip-count, and target timeout variables to the target script. In the standard archive, each container uses a one-CPU Docker quota, an 8GB memory limit, an 8GB swap limit, and `-init`; the controller records the container ID, waits with `docker wait`, and recovers one `tar.gz` archive per run. The launcher does not use `taskset`, `numactl`, or Docker `-cpuset-cpus`, so runs are quota-limited but not pinned to physical cores.

Within a target-fuzzer invocation, simultaneous containers equal `NUM_CONTAINERS`; a 10-run cell can therefore run as 10 one-CPU containers or as smaller batches. The aggregate artifact preserves selected terminal summaries but not the global batch schedule, so we cannot reconstruct co-scheduling across the planned primary matrix. Endpoint comparisons therefore use completed approximately 24-hour runs selected by the same rule, and wall-clock curves are descriptive traces under the same container quota rather than CPU-time-normalized throughput claims.

Port conflicts are avoided by container network namespaces. The non-MQTT target scripts bind the server to loopback inside each container and pass a container-local endpoint to AFLNet-style network mode: port 2200 for LightFTP, port 21 for bftpd, ProFTPD, and Pure-FTPd,

port 25 for Exim, port 8554 for Live555, UDP port 5060 for Kamailio, port 3689 for Forked-daapd, and port 8080 for Lighttpd1. These ports are not published to the host with Docker `-p`, so concurrent containers can reuse the same numeric port without cross-run conflicts. MQTT artifacts use separate container networks or generated worker ports, but the MQTT archive is excluded from performance conclusions: the LoopFuzz Mosquitto rows are much shorter than 24 hours, and the MBFuzzer archive exposes a different Mosquitto-only coverage surface without AFLNet-lineage `edges`.

Campaigns are bounded by the Unix `timeout` wrapper with a two-second kill grace and preserved exit status. Target reset is handled by AFLNet and, where available, per-target cleanup hooks such as `ftpclean`, `clean`, or `run_pjsip`. Offline coverage replay restarts or kills daemons with bounded cleanup commands before accumulating `gcovr` summaries. The launcher does not set a fixed AFL random seed, and the archive does not preserve per-run PRNG seeds. We therefore treat repeated runs as independent stochastic trials rather than bit-for-bit replays.

Run filtering and missing-data handling. The intended primary matrix is 10 runs per fuzzer-target pair. We retain completed runs with at least 1400 archived minutes and final `b_abs`/IPSM `edges`. If more than 10 eligible rows remain, the deterministic rule keeps the lowest 10 run IDs before inspecting endpoint values. This avoids best-run selection but not every bias, because run ID can correlate with calendar time, restarts, or archive quality. In the updated primary archive, all nine non-MQTT target-fuzzer cells contain 10 selected eligible runs. Table 3 reports the recoverable selected inventory; it is still not a complete raw-run ledger because the package does not preserve every pre-filter launch and exclusion record.

5.2. Baselines, Metrics, and Statistical Protocol

We used two baseline layers. The *primary* baselines were AFLNet as the response-derived stateful greybox baseline and ChatAFL as the direct LLM-assisted predecessor. This choice is intended to isolate the evaluated LoopFuzz strategy within the closest implementation lineage, not to replace stronger external state-aware fuzzers. StateAFL is a natural additional baseline, but the current archive does not contain same-budget StateAFL run summaries under the deterministic selection rule used for Table 5; adding it without rerunning would mix incomparable evidence. The *auxiliary* reproduced baseline was NSFuzz [33] on eight non-HTTP/non-MQTT targets. The current package preserves terminal NSFuzz count artifacts, but it does not preserve enough NSFuzz-side build, instrumentation, and coverage-script metadata to certify metric identity on every target. A separate MBFuzzer archive exists only for Mosquitto and contains worker-level `gcovr` branch coverage, not primary-format `run_summary.csv` rows or AFLNet-lineage `edges`. We therefore report NSFuzz and MBFuzzer artifacts sep-

arately as external diagnostics rather than as horizontal winners against the AFLNet-lineage columns.

The main dependent variables are final `b_abs` and final `edges`. For AFLNet, ChatAFL, and LoopFuzz, the `b_abs` field is the absolute branch-coverage count from the same run-summary and coverage-replay pipeline. LightFTP, for example, uses the same Dockerfile lineage, LightFTP commit `139af7c`, `gcovr` replay script, `fftp` target, and `ftpclean` reset step for the AFLNet-lineage systems. The `edges` field is the archived IPSM edge count and is used as a state-exploration diagnostic for AFLNet, ChatAFL, and LoopFuzz. For NSFuzz, the branch and edge fields are labeled as artifact counts unless the corresponding build and instrumentation path can be audited. The NSFuzz edge field is not semantically identical to the response-derived IPSM transition abstraction maintained by the AFLNet-lineage fuzzers, so the results separate branch coverage from state-exploration diagnostics.

For each primary target and metric, we summarize selected endpoint values with mean and sample SD. We also compute pairwise two-sided Mann-Whitney U tests comparing LoopFuzz against AFLNet and ChatAFL, followed by Benjamini-Hochberg FDR control across the 36 primary pairwise tests (nine targets, two metrics, and two baselines). Cliff’s δ reports effect size, with positive values favoring LoopFuzz. We use the per-run terminal vectors in the updated `run_summary.csv` files, after applying the same deterministic selection rule as Table 5. Exact tests are not always available because several rows contain ties, so the implementation uses SciPy’s tied-rank handling. We do not report crash-discovery claims because crash triage and root-cause validation are not normalized across the suite.

5.3. Policy-Ablation Design

The ablation design follows the switches present in the provided ablation archive. The archived matrix compares the evaluated full controller against four policy perturbations: disabling CEGR, disabling frontier scheduling, disabling hypothesis validation, and disabling adaptive triggering. This organization reflects the actual archive rather than an idealized orthogonal factorization.

The archive does not contain the most direct admission-boundary ablation. A complete design would include *LoopFuzz-no-admission*, which would preserve the same model calls, prompts, query budget, plateau triggers, and scheduling policy, but bypass the *P/U/R/G* admission checks and directly apply model suggestions. Without that condition, Table 10 cannot isolate the complete causal effect of the admission boundary. It instead tests whether the evaluated integrated control strategy is sensitive to removing several exposed sub-controls. We apply the same deterministic run-selection rule as the primary comparison: keep completed runs with at least 1400 minutes, and if more than 10 valid runs remain, keep the lowest 10 run IDs. Ablation cells with fewer than 10 eligible rows are retained only as archive-level descriptive comparisons; the

Table 3: Recoverable run-selection inventory for the primary AFLNet/ChatAFL-lineage comparison. Each fuzzer cell reports selected eligible runs over the planned 10-run primary matrix, followed by the number of planned rows not represented in the selected cache. The updated primary archive supplies 10 selected eligible runs for every non-MQTT target-fuzzer cell; the current package still does not preserve a complete raw pre-filter launch ledger.

Target	AFLNet	ChatAFL	LoopFuzz	Recoverable status
LightFTP	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
bftpd	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
ProFTPD	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Pure-FTPD	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Exim	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Live555	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Kamailio	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Forked-daapd	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Lighttpd1	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.

current package does not preserve enough failure metadata to classify every missing ablation row.

5.4. Trace Accounting and Cost Reporting

Because LoopFuzz is a closed-loop LLM-in-the-loop system, terminal coverage alone is not enough for mechanism accounting. A complete mechanism-level archive would preserve, for every model call and candidate, the prompt, raw reply, extracted candidate, $P/U/R/G$ decision, rejection reason, bounded-trial response, coverage/IPSM gain, refinement event, latency, and token counts. Those fields would support a per-target admission-control behavior table reporting model calls, candidate counts, parseability pass rate, response-grounded acceptance rate, state/coverage-gain pass rate, rejection-reason distribution, CEGR repair count, mean call latency, and token cost.

The archive available for this re-audit does not contain those normalized event logs. It contains terminal run summaries, trajectory CSVs, ablation summaries, and auxiliary NSFuzz artifacts, but not the raw `fuzzer_stats`, `stall-interactions`, normalized admission-event logs, or per-call token/latency records needed to reconstruct the mechanism table. Table 4 therefore reports the archive status of the requested mechanism-level statistics rather than fabricated values. The entries marked N/R are not zeros; they are not recoverable from the current artifact. As a result, semantic drift remains a motivating failure mode and an auditable example trace here, not a quantified cross-target rate.

The four data directories audited here also do not contain a normalized cross-target LLM-cost summary. We therefore do not make a cost-efficiency claim in this re-audited experimental section.

6. Results

Table 5 summarizes the terminal aggregates used for the primary AFLNet and ChatAFL lineage comparison. Figs. 3 and 4 visualize the same selected runs as per-target trajectories. This layout follows the protocol-level comparison view: each panel keeps one target on its own scale, so the curves show both endpoint separation and the time at which each fuzzer saturates. Table 6 reports the corrected

rank-test summary. The table should not be read as a ranking against all state-aware protocol fuzzers or as a no-admission ablation; it evaluates the integrated LoopFuzz strategy against its nearest AFLNet and ChatAFL predecessor chain.

The results below are outcome-level results, not mechanism-level admission statistics. Table 4 shows that the archive lacks the event logs needed to report model-call counts, candidate counts, $P/U/R/G$ pass rates, rejection reasons, CEGR repairs, latency, or token cost. We therefore limit empirical claims to final coverage and response-derived IPSM behavior under the selected runs.

6.1. RQ1: Whole-Campaign Effect

LoopFuzz is evaluated here as an integrated control policy within the AFLNet and ChatAFL lineage, not as an isolated rejection heuristic or a replacement for all state-aware fuzzers. Table 5 and Fig. 3 show that direct open-loop assistance is not uniformly robust under the selected archived runs. ChatAFL ties AFLNet on LightFTP, finishes below AFLNet on Exim in `b_abs`, is close to AFLNet on Forked-daapd, and improves several other targets. Under these conditions, improved generation alone does not consistently translate into stronger end-of-campaign coverage.

Within the primary AFLNet/ChatAFL lineage, LoopFuzz reaches the highest mean final `b_abs` on eight of nine targets; AFLNet remains highest on Forked-daapd, where LoopFuzz also has much larger variance. After BH correction, LoopFuzz is higher than AFLNet on `b_abs` for eight targets and lower on Forked-daapd, while its advantage over ChatAFL remains corrected-significant on five targets. The remaining positive mean separations over ChatAFL, such as bftpd, ProFTPD, and Kamailio, should be read as descriptive rather than inferential. Section 6.5 analyzes Forked-daapd directly.

Table 7 shows why NSFuzz artifacts are useful but not a homogeneous branch-coverage ranking. LightFTP reports 414.3 ± 9.8 NSFuzz artifact branches, while same-build AFLNet-lineage rows cluster around 71. Forked-daapd is also diagnostic: NSFuzz reports 2722.8 ± 44.6 , while AFLNet, ChatAFL, and LoopFuzz report 2326.3 ± 77.3 , 2315.3 ± 50.1 , and 2171.9 ± 226.0 . The package lacks enough NSFuzz-side build and coverage-script metadata to certify

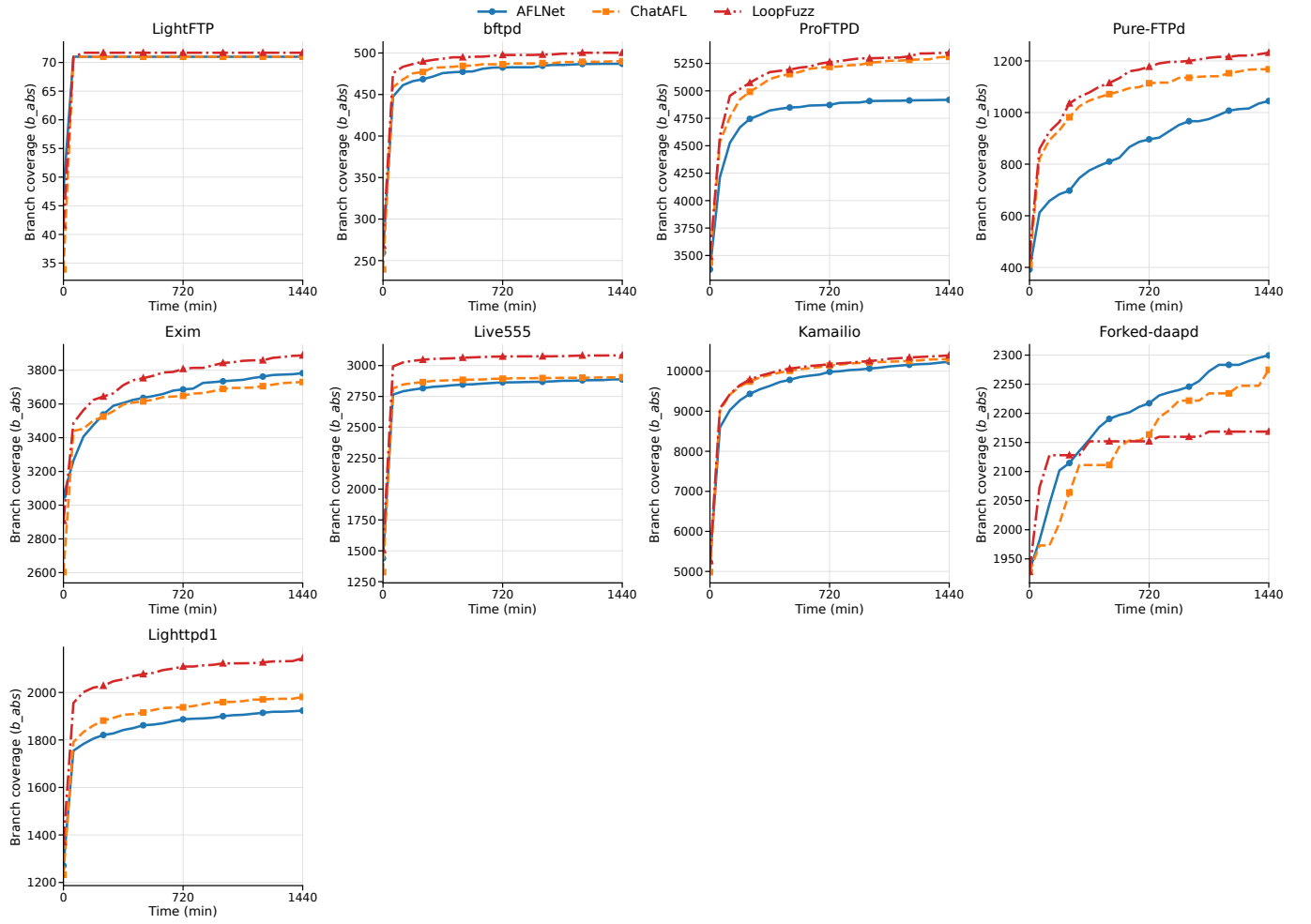


Figure 3: Branch-coverage trajectories for the nine primary-result targets. Each panel uses the same selected runs as Table 5.

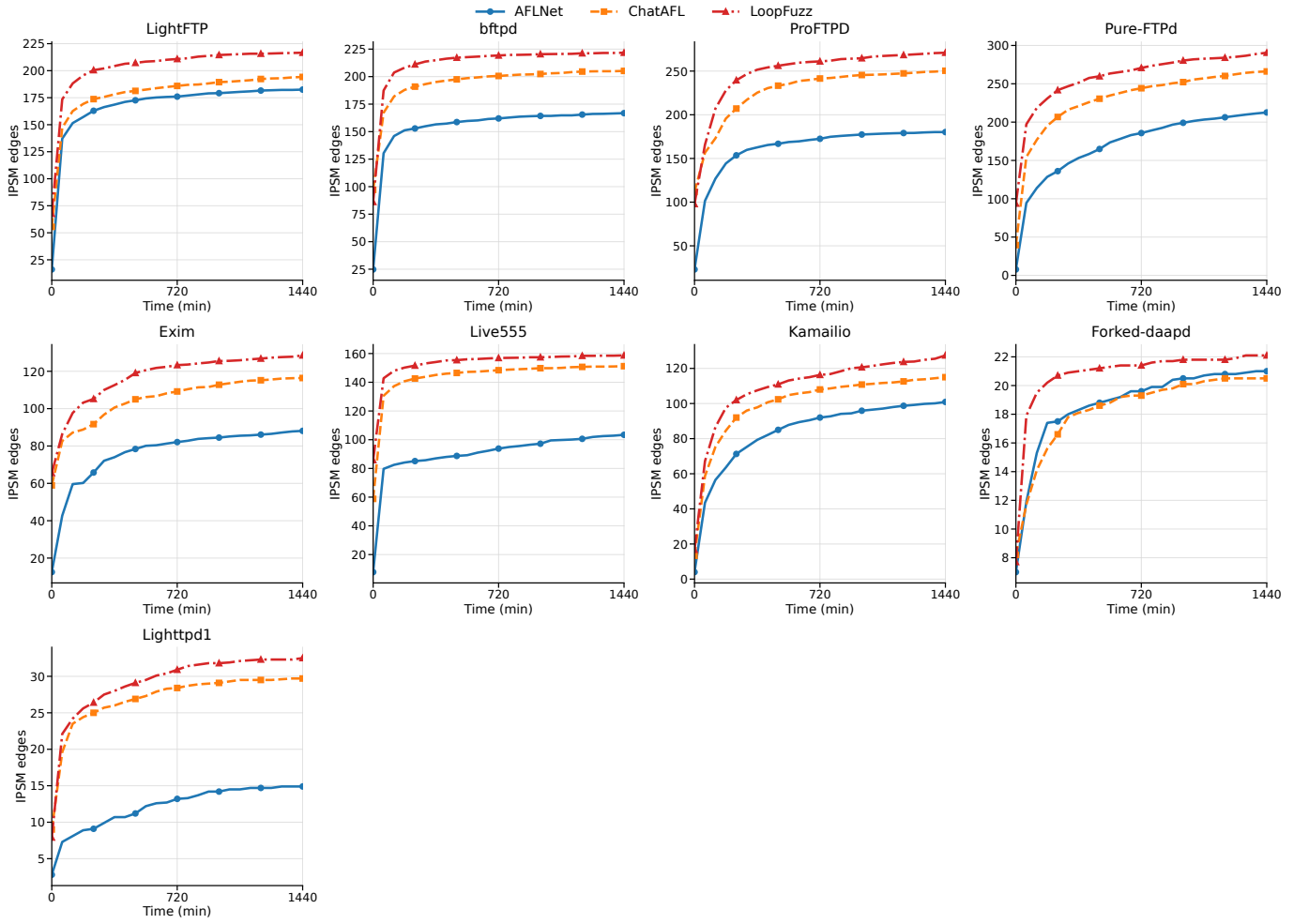


Figure 4: IPSM-edge exploration trajectories for the nine primary-result targets. Each panel uses the same selected runs as Table 5.

Table 4: Archive status for the requested per-target admission-control behavior statistics. N/R means not recoverable from the current run-summary archive or submission packages. The missing fields require event-level admission logs or `fuzzer_stats/stall-interactions` artifacts that are not present in the audited archive. These values are therefore excluded from empirical claims.

Target	Model calls	Candidates	<i>P</i> pass	<i>U</i> pass	<i>R</i> pass	<i>G</i> pass	Reject mix	CEGR repairs	Latency/tokens
LightFTP	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
bftpd	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
ProFTPD	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
Pure-FTPd	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
Exim	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
Live555	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
Kamailio	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
Forked-daapd	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R
Lighttpd1	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R	N/R

Table 5: Primary end-of-campaign results for `b_abs` and `edges` (mean $\pm$ sample SD over 10 selected completed runs per target-fuzzer cell). Bold marks the highest mean among AFLNet, ChatAFL, and LoopFuzz. Pairwise Mann-Whitney/BH/Cliff statistics for LoopFuzz versus the two baselines are summarized in Table 6.

Proto.	Target	Final <code>b_abs</code>			Final <code>edges</code>		
		AFLNet	ChatAFL	LoopFuzz	AFLNet	ChatAFL	LoopFuzz
FTP	LightFTP	71.0 $\pm$ 0.0	71.0 $\pm$ 0.0	71.7$\pm$1.6	182.8 $\pm$ 10.2	194.2 $\pm$ 10.2	216.6$\pm$10.4
FTP	bftpd	488.6 $\pm$ 13.9	492.3 $\pm$ 15.1	501.5$\pm$7.4	166.9 $\pm$ 9.4	205.2 $\pm$ 9.7	221.7$\pm$16.9
FTP	ProFTPD	4919.9 $\pm$ 97.8	5317.3 $\pm$ 101.7	5358.8$\pm$136.1	180.5 $\pm$ 23.1	250.5 $\pm$ 9.1	271.7$\pm$18.8
FTP	Pure-FTPd	1054.3 $\pm$ 76.4	1179.2 $\pm$ 31.0	1242.6$\pm$30.6	213.9 $\pm$ 11.9	266.2 $\pm$ 14.3	291.0$\pm$23.6
SMTP	Exim	3794.5 $\pm$ 42.4	3746.2 $\pm$ 47.7	3908.5$\pm$43.2	88.3 $\pm$ 5.3	116.4 $\pm$ 10.2	128.5$\pm$7.8
RTSP	Live555	2894.1 $\pm$ 23.3	2911.1 $\pm$ 10.9	3088.7$\pm$22.4	103.9 $\pm$ 10.1	151.2 $\pm$ 3.9	158.7$\pm$8.5
SIP	Kamailio	10244.0 $\pm$ 118.1	10314.7 $\pm$ 98.4	10402.7$\pm$141.3	101.1 $\pm$ 7.7	115.7 $\pm$ 13.9	127.6$\pm$9.6
DAAP/DACP	Forked-daapd	2326.3$\pm$77.3	2315.3 $\pm$ 50.1	2171.9 $\pm$ 226.0	21.1 $\pm$ 1.3	20.5 $\pm$ 2.1	22.1$\pm$1.5
HTTP	Lighttpd1	1930.4 $\pm$ 18.7	1987.5 $\pm$ 38.3	2156.5$\pm$111.8	14.9 $\pm$ 3.8	29.7 $\pm$ 4.7	32.5$\pm$10.8

Table 6: Pairwise statistical summary for LoopFuzz against each primary baseline. Each cell reports raw *p* / BH-adjusted *q* / Cliff’s δ for the two-sided Mann-Whitney U test on selected per-run endpoints; positive δ favors LoopFuzz and negative values favor the baseline.

Target	<code>b_abs</code>		<code>edges</code>	
	vs. AFLNet	vs. ChatAFL	vs. AFLNet	vs. ChatAFL
LightFTP	0.078/0.093/ + 0.30	0.078/0.093/ + 0.30	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 0.94
bftpd	0.017/0.030/ + 0.64	0.150/0.169/ + 0.39	< 0.001/ < 0.001/ + 1.00	0.031/0.043/ + 0.58
ProFTPD	< 0.001/ < 0.001/ + 1.00	0.623/0.623/ + 0.14	< 0.001/ < 0.001/ + 1.00	0.012/0.023/ + 0.67
Pure-FTPd	< 0.001/ < 0.001/ + 1.00	0.001/0.003/ + 0.86	< 0.001/ < 0.001/ + 1.00	0.013/0.023/ + 0.67
Exim	0.001/0.002/ + 0.90	< 0.001/ < 0.001/ + 0.98	< 0.001/ < 0.001/ + 1.00	0.025/0.037/ + 0.60
Live555	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 1.00	0.034/0.045/ + 0.57
Kamailio	0.021/0.035/ + 0.62	0.345/0.365/ + 0.26	< 0.001/ < 0.001/ + 0.97	0.049/0.063/ + 0.53
Forked-daapd	0.026/0.037/ - 0.60	0.026/0.037/ - 0.60	0.164/0.179/ + 0.37	0.113/0.132/ + 0.42
Lighttpd1	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 0.95	< 0.001/ < 0.001/ + 0.98	0.596/0.613/ + 0.15

Table 7: Auxiliary NSFuzz branch-count artifacts. This table deliberately omits AFLNet, ChatAFL, and LoopFuzz columns; their same-build `b_abs` values appear in Table 5. The current package does not preserve enough NSFuzz-side build, instrumentation, and coverage-script metadata to certify metric identity on every target, so these NSFuzz counts are retained as external artifact diagnostics rather than as horizontal branch-coverage winners.

Target	NSFuzz artifact branch count
LightFTP	414.3 $\pm$ 9.8
bftpd	477.6 $\pm$ 3.9
ProFTPD	5335.0 $\pm$ 86.9
Pure-FTPd	1332.2 $\pm$ 8.8
Exim	4191.5 $\pm$ 62.0
Live555	3021.5 $\pm$ 42.1
Kamailio	10635.3 $\pm$ 123.8
Forked-daapd	2722.8 $\pm$ 44.6

metric identity, so these rows bound the claim rather than declare a cross-tool winner.

Table 8: State-exploration diagnostics in the auxiliary archive. AFLNet, ChatAFL, and LoopFuzz report final response-derived AFLNet-lineage **edges**. The NSFuzz column reports *NSFuzz artifact edge count* and is not horizontally comparable to those IPSM edge counts. Bold marks the highest mean only among AFLNet-lineage columns.

Target	AFLNet edges	ChatAFL edges	LoopFuzz edges	NSFuzz artifact edge count
LightFTP	182.8±10.2	194.2±10.2	216.6±10.4	11.0±0.0
bftpd	166.9±9.4	205.2±9.7	221.7±16.9	48.7±4.8
ProFTPD	180.5±23.1	250.5±9.1	271.7±18.8	186.0±11.5
Pure-FTPd	213.9±11.9	266.2±14.3	291.0±23.6	17.6±0.5
Exim	88.3±5.3	116.4±10.2	128.5±7.8	13.9±1.2
Live555	103.9±10.1	151.2±3.9	158.7±8.5	49.8±2.0
Kamailio	101.1±7.7	115.7±13.9	127.6±9.6	14.3±0.7
Forked-daapd	21.1±1.3	20.5±2.1	22.1±1.5	27.0±6.1

Table 9: MQTT/Mosquitto artifact-integrity summary. This table is not a performance ranking: the AFLNet row is a 24-hour AFLNet-lineage run, the LoopFuzz Mosquitto rows are much shorter, and MBFuzzer exposes **gcovr** branch coverage from a Mosquitto-only worker archive with no AFLNet-lineage **edges**.

Artifact	Runs	Runtime (min)	Branch metric	Edge metric	Use in this paper
AFLNet Mosquitto	10	1479.0±0.0	b_abs 1814.3±147.3	edges 36.0±4.5	Same lineage but no ChatAFL row; excluded from the primary three-way table.
LoopFuzz Mosquitto	10	137.4±134.7	b_abs 2518.5±269.9	edges 107.7±18.1	Not same-budget with AFLNet; excluded from performance conclusions.
MBFuzzer Mosquitto	10	≈ 1480	gcovr branches 2827.0±58.9	N/A	External Mosquitto-only artifact; no primary-format b_abs or AFLNet-lineage edges .

The LightFTP anomaly is especially instructive. The AFLNet-lineage LightFTP runs share the same target commit, **gcovr** replay script, **fftp** binary, and **ftpclean** reset path, yet branch coverage saturates near 71 within the first hour. The NSFuzz artifact is almost six times larger. Without NSFuzz-side build and replay metadata, the archive cannot distinguish stronger exploration from a different coverage surface. Section 6.4 records the audit evidence and keeps this row out of the primary performance conclusion.

6.2. RQ2: IPSM Escape Dynamics

Terminal **edges** provide the clearest primary-lineage evidence for improved state exploration. In the AFLNet/ChatAFL comparison, LoopFuzz attains the highest mean on all nine primary targets, as visualized in Fig. 4. After BH correction, the LoopFuzz-versus-AFLNet edge advantage remains significant on eight targets; Forked-daapd remains positive by mean but not significant. Against ChatAFL, the corrected edge advantage remains significant on six targets, while Kamailio, Forked-daapd, and Lighttpd1 are positive by mean but not corrected-significant.

Table 8 keeps the NSFuzz edge archive separate from this state-exploration claim. AFLNet, ChatAFL, and LoopFuzz report response-derived IPSM edges, whereas the NSFuzz column is an artifact edge count from a different state-aware implementation. We therefore do not select an NSFuzz-versus-LoopFuzz edge winner from that table. The diagnostic mismatch is visible in the data: on LightFTP, NSFuzz has a much larger branch-count artifact in Table 7 but an artifact edge count of only 11.0±0.0; on Forked-daapd, NSFuzz reports 27.0±6.1 artifact edges while the AFLNet-lineage edge counts are 21.1±1.3, 20.5±2.1, and

22.1±1.5. These values may be useful for reproducing the archive, but they are not a common semantic edge metric.

6.3. RQ3: End-of-Campaign Outcome

The **b_abs** comparison is less uniform even before considering external artifacts. In the primary three-way comparison, LoopFuzz is strongest by mean on most targets but does not lead Forked-daapd. The NSFuzz branch-count artifacts add boundary evidence rather than a homogeneous ranking: the LightFTP and Forked-daapd artifact counts are much larger than the AFLNet-lineage branch counts, but the archive does not prove that the same build, instrumentation, replay harness, and coverage extraction surface were used. Table 9 shows a different boundary for Mosquitto: MBFuzzer has a recoverable **gcovr** branch count, but no AFLNet-lineage **b_abs/edges**, and the LoopFuzz Mosquitto rows are much shorter than 24 hours. The **edges** result has a narrower scope: it supports improved response-derived IPSM exploration within the AFLNet/ChatAFL lineage, but the NSFuzz artifact edge counts and MBFuzzer Mosquitto artifacts cannot extend or negate that claim because they are not the same measurement. This distinction matters for interpretation: the audited archive supports improved state exploration within the closest lineage and a selective absolute-branch-coverage advantage on the primary targets, not a blanket dominance claim over specialized fuzzers.

6.4. Artifact Anomaly: LightFTP

LightFTP is the largest metric-consistency warning in the auxiliary archive. In the primary table, all three AFLNet-lineage systems use the same LightFTP build and coverage-replay path. The local Dockerfile

checks out LightFTP commit 139af7c; the fuzzing build uses `AFL_USE_ASAN=1 CC=afl-clang-fast`; the coverage build applies the local `gcov.patch` and compiles with `-fprofile-arcs -ftest-coverage`; and the coverage script replays queue entries against `fftp`, runs `gcovr -r .`, and resets the shared FTP directory through `ftpclean`. Under that path, the branch-coverage trajectory reaches 71.0 by 60 minutes for AFLNet and ChatAFL, and 71.7 by 120 minutes for LoopFuzz, then remains flat through 1440 minutes.

The NSFuzz artifact for the same nominal target is 414.3 ± 9.8 branch-count units. The current archive does not include the NSFuzz-side LightFTP Dockerfile, target commit confirmation, patch set, compiler and coverage flags, replay harness, or coverage extraction script. Because those fields are missing, we cannot tell whether the NSFuzz artifact reflects a stronger exploration strategy on the identical LightFTP coverage surface or a different target build, instrumentation scope, input model, or counting script. The accompanying NSFuzz artifact edge count is also only 11.0 ± 0.0 , which reinforces that the NSFuzz state/edge artifact is not the AFLNet-lineage response-derived IPSM metric.

We therefore make a conservative choice. LightFTP remains in Table 5 for the AFLNet/ChatAFL/LoopFuzz lineage comparison, where the build and coverage pipeline are shared. The NSFuzz LightFTP value remains in Table 7 as an external artifact anomaly that future work should re-run under a common Dockerfile and coverage script. It is not used as evidence that NSFuzz opened extra branches in the identical LightFTP build, nor as evidence that LoopFuzz is inferior to NSFuzz on a certified common branch-coverage metric.

6.5. Failure-Target Analysis: Forked-daapd

Forked-daapd is the clearest primary failure target for LoopFuzz. In Table 5, LoopFuzz ends at 2171.9 ± 226.0 final `b_abs`, below AFLNet at 2326.3 ± 77.3 and ChatAFL at 2315.3 ± 50.1 . The variance is also diagnostic: LoopFuzz’s branch-coverage standard deviation is about three times AFLNet’s and more than four times ChatAFL’s. Table 7 also reports a high NSFuzz artifact branch count of 2722.8 ± 44.6 for this target, but we do not treat that value as a certified same-pipeline comparison. The same target therefore bounds both claims: LoopFuzz can improve response-derived IPSM exploration while still missing branch coverage reached by the simpler AFLNet-lineage baseline, and the external artifact suggests that stronger state-aware strategies may expose additional behavior under conditions that require re-audit.

The trajectory evidence points to a state-signal mismatch rather than a slow start. LoopFuzz is ahead after 60 minutes on Forked-daapd (2072.5 `b_abs`) while AFLNet and ChatAFL are at 1981.6 and 1973.1. It then plateaus at 2168.6 from 1080 to 1440 minutes, while AFLNet rises from 2272.3 to 2299.5 and ChatAFL from 2234.3 to 2274.7. The edge trajectory has the opposite shape: LoopFuzz

reaches 17.9 IPSM edges by 60 minutes, versus 11.9 and 11.7, and ends slightly higher at 22.1. Thus, faster response-derived state growth does not translate into deeper branch exploration on this target.

A plausible explanation is that DAAP/DACP exposes a noisy or sparse response signal for the AFLNet-lineage IPSM abstraction. Forked-daapd contains media-library and control-session behavior where requests can change visible responses without exercising branches that dominate `b_abs`. LoopFuzz may therefore promote candidates that look state-progressive but are not branch-productive. The high variance is also compatible with initialization or early-history instability, although the archive lacks per-run server-state logs to confirm that mechanism.

The ablation row does not support a simple “one switch is too strict” explanation. On Forked-daapd, removing CEGR, frontier scheduling, hypothesis validation, or adaptive triggering all lowers `b_abs` relative to the full controller (-104.4 , -145.0 , -129.9 , and -91.8). Some relaxed variants slightly increase `edges`, but not branch coverage. This means the failure is unlikely to be fixed merely by disabling one exposed sub-control. More likely, the target needs a different admission signal, such as richer DAAP/DACP response parsing, target-state normalization, database/session initialization checks, or a *LoopFuzz-no-admission* run to test whether model suggestions are being rejected too aggressively or are simply unhelpful for this protocol.

6.6. Ablation: Boundary-Condition Analysis

Table 10 reports the archived policy perturbations as deltas relative to the evaluated full controller. Each cell shows $\Delta \text{b_abs} / \Delta \text{edges}$. These rows should not be read as a no-admission experiment: none preserves the same model-call path while bypassing the full *P/U/R/G* admission formula and directly accepting model suggestions. The updated archive therefore supports only an integrated-policy interpretation.

Within that scope, the ablation table is best read as a boundary-condition analysis rather than as a monotone proof that every switch is always beneficial. The clearest stable signal is frontier control: removing it reduces mean `b_abs` on all nine targets, which suggests that state-directed budget allocation is broadly useful in this archive. The remaining switches are more target-dependent.

CEGR is the main non-monotone case. Removing CEGR increases `b_abs` on ProFTPD by $+99.1$ and on Live555 by $+1.2$, while reducing `edges` on both targets (-5.2 and -1.5). This pattern is consistent with CEGR sometimes over-constraining the local message space: a repaired hypothesis can prevent repeated invalid continuations, but it can also narrow a target-specific region where looser mutations still reach additional instrumented branches. On the other seven targets, removing CEGR reduces `b_abs`, so the archive does not show that CEGR is generally harmful. It shows a boundary condition: local repair helps most primary targets, but ProFTPD and Live555 may need a

less aggressive repair policy or a decay mechanism that reopens constraints after they stop producing coverage.

Adaptive triggering shows a different boundary condition. Removing it improves both **b_abs** and **edges** on Pure-FTPd (+6.3/+10.7) and Kmailio (+26.9/+5.2), and improves **b_abs** but not **edges** on Live555 (+5.0/-3.6). For LightFTP, ProFTPD, and Lighttpd1, the same switch raises **edges** but lowers **b_abs**. These mixed deltas suggest that the plateau trigger can be too late or too conservative for targets where earlier model intervention exposes useful request variants, but that more intervention can also expand response-derived state structure without translating into branch coverage. Because the current archive lacks per-target model-call and admission-event counts, we treat this as a diagnostic interpretation rather than as a quantified mechanism claim.

Disabling hypothesis validation has the same cautionary shape: **b_abs** drops on every target, yet **edges** rises on Pure-FTPd, Kmailio, and Forked-daapd. This reinforces that a larger response-derived IPSM is not automatically a better campaign endpoint. The complete strategy is therefore reasonably robust as an integrated policy, especially for branch coverage under frontier control, but individual components remain target-dependent. A future *LoopFuzz-no-admission* run would still be required to isolate whether the full admission boundary, holding model calls and scheduling fixed, is the dominant causal factor.

7. Discussion

The paper makes one central design claim. In stateful protocol fuzzing, LLM generation should be governed by runtime evidence before it changes a campaign. ChatAFL already showed that an LLM can generate protocol text [18]. LoopFuzz asks a different question: when should such output be trusted by the campaign? Our design answer is closed-loop runtime admission control. It checks structure and parseability before a trial, then uses the trial response, state movement, and coverage gain to decide whether an LLM suggestion can strengthen the queue. The present archive supports this design as part of an integrated control strategy, but it does not contain the no-admission ablation needed to prove the admission boundary’s independent effect.

The admission controller and the scheduler solve different parts of the same problem. Admission control filters low-value suggestions. It does not decide where to spend the recovered budget. State-aware scheduling decides where to search. Without closed-loop intervention, however, it still depends on ordinary mutation to cross semantic bottlenecks. LoopFuzz combines the two because admission control and state-directed search reinforce each other.

The results also show why response-derived state counts must be read carefully. A campaign can accumulate superficially different rejection paths without moving toward high-value logic. LoopFuzz therefore reads state exploration together with **b_abs** and demotes states that remain

unproductive over repeated selections. In this setting, productive states matter more than raw state counts. For that reason, **edges** is best interpreted as a companion diagnostic for campaign behavior, not as a substitute for absolute branch coverage.

Forked-daapd is the strongest example of this limitation. LoopFuzz grows the response-derived IPSM faster and ends with slightly more primary-lineage **edges**, but it underperforms AFLNet and ChatAFL in final branch coverage; the NSFuzz artifact branch count is also much higher and should be re-audited under a common pipeline. This outcome suggests that DAAP/DACP response-level progress can be too weak or noisy for the current admission and scheduling policy. It also shows why the paper’s claim must remain scoped to the evaluated lineage and outcome mix rather than being framed as universal improvement.

The present method is evaluated on textual or semi-textual protocols whose message regions can be parsed cheaply and whose responses provide useful progress signals. That scope covers the benchmark suite used here and many request-response protocols of practical interest. Opaque binary or encrypted protocols require stronger parsers, auxiliary instrumentation, or richer response abstractions. Response codes alone may not expose state reachability in those settings. That extension matters, but it lies beyond the current empirical scope.

The present evaluation is also centered on campaign-control behavior rather than on vulnerability yield. We preserve crashes and queue artifacts, but we do not report new CVEs or newly confirmed bugs in this study because crash deduplication, exploitability review, and cross-target triage remain incomplete under one uniform protocol. The paper should therefore be read as evidence about campaign control and exploration quality, not as an end-to-end demonstration of stronger bug-finding yield.

Counterexample-guided local refinement is also a deliberate design choice. Repeated unconstrained regeneration does not accumulate persistent knowledge. Local refinement turns failed executions into stored evidence. It makes runtime evidence cumulative rather than disposable. The boundary-condition analysis, however, shows that this repair pressure can be target-dependent. On ProFTPD and Live555, disabling CEGR raises **b_abs** while lowering **edges**, which suggests that local repairs may sometimes over-shrink a search region that still contains branch-productive variants. Adaptive triggering has a related boundary condition: on Pure-FTPd and Kmailio, earlier or less conservative intervention appears to help both reported endpoints, whereas on other targets it mainly changes response-derived state structure. These observations do not invalidate the integrated controller, but they do prevent a stronger claim that each component is independently necessary on every protocol. The complete strategy is best characterized as broadly robust with target-dependent sub-controls.

Table 10: Policy-perturbation results relative to the evaluated full controller. The table does not include a *LoopFuzz-no-admission* condition that preserves model calls and scheduling while bypassing the full *P/U/R/G* admission checks. The first number in each cell is final **b_abs**; the second is final **edges**. The Full column reports mean final **b_abs** / **edges**; other cells report deltas relative to that mean. Runs with archived runtime below 1400 minutes are excluded; when more than 10 valid runs remain, we keep the lowest 10 run IDs. Some ablated target-policy cells contain seven to nine eligible runs after this filter.

Target	Full	No CEGR	No frontier	No hypothesis	No adaptive trigger
LightFTP	71.6 / 213.9	-0.2 / -13.5	-0.6 / -14.5	-0.5 / -11.0	-0.6 / +1.8
bftpd	501.5 / 221.7	-3.1 / -7.5	-3.0 / -8.4	-5.2 / -5.0	-7.0 / -12.1
ProFTPD	5358.8 / 271.7	+99.1 / -5.2	-118.7 / -37.3	-8.3 / +0.1	-20.6 / +0.4
Pure-FTPD	1242.6 / 291.0	-19.2 / +1.1	-18.9 / -1.4	-0.6 / +15.9	+6.3 / +10.7
Exim	3908.5 / 128.5	-29.0 / -2.8	-40.7 / -2.8	-43.0 / -3.3	-40.6 / -8.9
Live555	3088.7 / 158.7	+1.2 / -1.5	-21.0 / -9.6	-12.2 / -4.4	+5.0 / -3.6
Kamailio	10402.7 / 127.6	-96.7 / +2.5	-121.4 / -5.0	-100.6 / +1.9	+26.9 / +5.2
Forked-daapd	2171.9 / 22.1	-104.4 / -0.9	-145.0 / +0.1	-129.9 / +0.7	-91.8 / -1.7
Lighttpd1	2156.5 / 32.5	-62.4 / -3.3	-15.9 / +1.9	-33.9 / -2.4	-27.1 / +4.1

8. Threats to Validity

Internal validity: LLM-assisted fuzzing remains sensitive to generation variance, target nondeterminism, and timing effects. We mitigate these threats by fixing the environment, pinning the model endpoint, using low temperatures for grammar extraction and refinement, and using repeated 24-hour runs. Each archived run is isolated in a Docker container with a one-CPU quota and separate network namespace. However, the artifact lacks the global co-scheduling plan, host-load trace, Docker **stats**, CPU-frequency trace, and per-run PRNG seeds. It therefore cannot rule out residual wall-clock effects from contention, storage pressure, network jitter, or scheduler decisions. Growth-rate plots are descriptive wall-clock traces, not CPU-time-normalized throughput claims. Plateau triggering and refinement are history-dependent, so the study cannot cleanly separate model variance from control-policy variance.

Statistical conclusion validity: The primary table spans nine targets, two endpoint metrics, and two LoopFuzz-versus-baseline comparisons per metric. Table 6 reports raw Mann-Whitney *p* values, BH-adjusted *q* values, and Cliff’s δ for all 36 tests from the selected per-run endpoint vectors. These tests are still limited by small sample size ($n = 10$ per cell), ties in some endpoint distributions, and the fact that the deterministic run-selection rule is based on recoverable archive rows rather than a complete launch ledger. We therefore interpret corrected-significant rows as endpoint evidence within the audited archive, not as universal target behavior.

External validity: The benchmark suite spans 10 implementations across seven protocol families, but the primary three-way comparison covers the nine non-MQTT targets with comparable AFLNet, ChatAFL, and LoopFuzz rows. Evidence is strongest for textual or semi-textual protocols whose progress is visible in responses and for systems sharing the AFLNet/ChatAFL response-derived IPSM lineage. We do not claim unchanged transfer to opaque binary or encrypted protocols, nor superiority over all state-aware

fuzzers. The archive lacks same-format StateAFL, Bleem, and Logos reproductions under the same selection rule. NSFuzz artifact rows are target-specific diagnostics rather than a unified cross-system state-machine comparison, and MBFuzzer is available only as a Mosquito worker archive without AFLNet-lineage **edges**.

Construct validity: We use final **b_abs** and final **edges** as proxies for fuzzing effectiveness, but neither fully captures semantic protocol progress. AFLNet-lineage **b_abs** rows use shared target and replay pipelines, while NSFuzz branch counts remain artifact counts unless their build, instrumentation, and coverage extraction can be audited; the LightFTP 414.3 ± 9.8 artifact shows why this distinction matters. Runtime admission means structured output checks, bounded validation executions, response-grounded acceptability, state-reachability signals, and CEGR, not formal verification or a perfect pre-execution oracle. The ablation archive also lacks *LoopFuzz-no-admission*, and event-level admission logs are unavailable. Semantic drift is therefore specified operationally and illustrated by a trace, not quantified as a cross-target prevalence or acceptance rate.

Reproducibility validity: ChatAFL and LoopFuzz share the same pinned model endpoint, each primary target is drawn from one declared archive, and the paper states the deterministic run-selection rule. Scripts document container isolation, CPU and memory quotas, internal ports, timeout handling, result manifests, and recovery logic. Table 3 reports the recoverable selected inventory, but the artifact lacks full pre-filter inventories and failure metadata for any runs not represented in the selected cache. Mosquito/MQTT rows are excluded from performance conclusions because they lack a homogeneous comparison surface: LoopFuzz averages 137.4 minutes, AFLNet and MBFuzzer are close to 1480 minutes, and MBFuzzer exposes worker-level **gcovr** branch coverage rather than AFLNet-lineage **b_abs/edges**. A complete package should also preserve fuzzer statistics, stall-interaction logs, raw prompts and replies, admission decisions, rejection reasons,

refinement events, latency, token counters, exclusion logs, launch manifests, batch schedules, resource traces, and PRNG seeds.

9. Conclusion

This evaluation supports a bounded design lesson: fluent protocol text is insufficient for a stateful campaign. In the evaluated AFLNet and ChatAFL lineage, the integrated LoopFuzz control strategy combines runtime admission, local refinement, and state-aware scheduling to reduce the risk of semantically premature model suggestions.

Under the audited primary AFLNet and ChatAFL lineage comparison, LoopFuzz’s clearest effect is improved state exploration, with a selective branch-coverage advantage on the same benchmark archive. Mean final response-derived IPSM **edges** improves across the nine off-the-shelf primary targets; after correction, the advantage remains significant on eight targets versus AFLNet and six targets versus ChatAFL. Mean final **b_abs** improves on eight of nine primary targets versus AFLNet and on eight of nine versus ChatAFL by mean, with corrected significance on eight and five targets respectively. Forked-daapd remains the primary-target exception for branch coverage.

The broader baseline picture is mixed and incomplete. NSFuzz artifact counts are informative external diagnostics, especially on LightFTP and Forked-daapd, but the current package does not preserve enough NSFuzz-side build, instrumentation, replay, and coverage-extraction metadata to treat those rows as homogeneous **b_abs** rankings. MBFuzzer is recoverable only as a Mosquitto worker artifact with **gcovr** branch coverage and no AFLNet-lineage **edges**. Forked-daapd is the clearest primary negative target: LoopFuzz shows faster early response-derived state growth there, but its branch coverage plateaus below AFLNet and ChatAFL; the higher NSFuzz artifact count should be re-audited under a common pipeline. Same-format StateAFL, Bleem, and Logos reproductions are not available in the current archive. The archived ablations show a robust integrated strategy but target-dependent sub-controls: CEGR can over-constrain ProFTPD and Live555 branch exploration, and adaptive triggering can be too conservative on Pure-FTPd and Kamailio. They also lack the *LoopFuzz-no-admission* condition needed to isolate the full admission boundary. The evidence therefore supports a precise conclusion: the integrated LoopFuzz control strategy improves state-exploration behavior within the closest AFLNet/ChatAFL lineage and improves absolute branch coverage on most primary targets, but this study does not establish the independent causal effect of the complete admission formula, per-module necessity on every target, or dominance over specialized external fuzzers.

CRedit authorship contribution statement

Author contribution roles must be confirmed by all authors before submission. A conservative draft is maintained in `AUTHOR_DECLARATIONS_TO_CONFIRM.md` for transfer into Editorial Manager or into this section after confirmation.

Declaration of competing interest

The competing-interest declaration must be confirmed by all authors before submission. A draft declaration is maintained in `AUTHOR_DECLARATIONS_TO_CONFIRM.md`.

Funding

Funding information must be confirmed by all authors before submission. A draft funding statement is maintained in `AUTHOR_DECLARATIONS_TO_CONFIRM.md`.

Data availability

Data and code availability statements must be confirmed before submission. The current artifact contains local run summaries and analysis scripts, but no public repository URL or DOI is declared in this manuscript.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI Codex for language editing, LaTeX formatting, and submission-compliance checking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] V.-T. Pham, M. B'ohme, A. Roychoudhury, AFLNet: A greybox fuzzer for network protocols, in: 2020 IEEE 13th International Conference on Software Testing, Verification and Validation (ICST), 2020, pp. 460–465. doi:10.1109/ICST46399.2020.00062.
- [2] R. Natella, StateAFL: Greybox fuzzing for stateful network servers, Empirical Software Engineering 27 (2022). doi:10.1007/s10664-022-10233-3, art. no. 191.
- [3] J. Ba, M. B'ohme, Z. Mirzamomen, A. Roychoudhury, Stateful greybox fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3255–3272.
- [4] F. Hu, J. Ji, H. Shu, Z. Li, T. Liu, C. Zhang, Formatted stateful greybox fuzzing of TLS server, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), 2024, pp. 151–160. doi:10.1109/icst60714.2024.00022.
- [5] Q. Zhang, X. Bai, X. Li, H. Duan, Q. Li, Z. Li, Resolver-fuzz: Automated discovery of DNS resolver vulnerabilities with query-response fuzzing, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4729–4746.
- [6] X. Song, J. Wu, Y. Zeng, H. Pan, C. Zuo, Q. Zhao, S. Guo, MBFuzzer: A multi-party protocol fuzzer for MQTT brokers, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 6179–6197.

- [7] Z. Luo, J. Yu, F. Zuo, J. Liu, Y. Jiang, T. Chen, A. Roychoudhury, J. Sun, Bleem: Packet sequence oriented fuzzing for protocol implementations, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 4481–4498.
- [8] F. Wu, Z. Luo, Y. Zhao, Q. Du, J. Yu, R. Peng, H. Shi, Y. Jiang, Logos: Log guided fuzzing for protocol implementations, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024, pp. 1720–1732. doi:10.1145/3650212.3680394.
- [9] N. Redini, A. Continella, D. Das, G. D. Pasquale, N. Spahn, A. Machiry, A. Bianchi, C. Kruegel, G. Vigna, DIANE: Identifying fuzzing triggers in apps to generate under-constrained inputs for iot devices, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 484–500. doi:10.1109/SP40001.2021.00066.
- [10] Y. Dong, S. M. M. Rashid, A. Ranjbar, T. Wu, S. R. Hussain, M. S. Mahmud, A. A. Ishtiaq, K. Tu, T. Yang, CoreCrisis: Threat-guided and context-aware iterative learning and fuzzing of 5g core networks, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 5287–5306.
- [11] M. B'ohme, V.-T. Pham, M.-D. Nguyen, A. Roychoudhury, Directed greybox fuzzing, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2329–2344. doi:10.1145/3133956.3134020.
- [12] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, A. L. Hosking, Seed selection for successful fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 230–243. doi:10.1145/3460319.3464829.
- [13] M. Wu, L. Jiang, J. Xiang, Y. Huang, H. Cui, L. Zhang, Y. Zhang, One fuzzing strategy to rule them all, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1634–1645. doi:10.1145/3510003.3510174.
- [14] X. Zhu, M. B'ohme, Regression greybox fuzzing, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 2169–2182. doi:10.1145/3460120.3484596.
- [15] J. Ba, G. J. Duck, A. Roychoudhury, Efficient greybox fuzzing to detect memory errors, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12. doi:10.1145/3551349.3561161.
- [16] R. Natella, V.-T. Pham, ProFuzzBench: A benchmark for stateful protocol fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 662–665. doi:10.1145/3460319.3469077.
- [17] A. Andronidis, C. Cadar, SnapFuzz: high-throughput fuzzing of network applications, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 340–351. doi:10.1145/3533767.3534376.
- [18] R. Meng, M. Mirchev, M. B'ohme, A. Roychoudhury, Large language model guided protocol fuzzing, in: 31st Annual Network and Distributed System Security Symposium (NDSS 2024), 2024, pp. 1–17. doi:10.14722/ndss.2024.24556.
- [19] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, L. Zhang, Fuzz4All: Universal fuzzing with large language models, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13. doi:10.1145/3597503.3639121.
- [20] J. Wang, L. Yu, X. Luo, LLMIF: Augmented large language model for fuzzing IoT devices, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 881–896. doi:10.1109/sp54263.2024.00211.
- [21] X. Ma, L. Luo, Q. Zeng, From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter IoT devices, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4783–4800.
- [22] D. Wang, Y. Li, Z. Zhang, K. Chen, CarpetFuzz: Automatic program option constraint extraction from documentation for fuzzing, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 1919–1936.
- [23] Y. Lyu, Y. Xie, P. Chen, H. Chen, Prompt fuzzing for fuzz driver generation, in: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 3793–3807. doi:10.1145/3658644.3670396.
- [24] Y. Deng, C. S. Xia, H. Peng, C. Yang, L. Zhang, Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 423–435. doi:10.1145/3597926.3598067.
- [25] Y. Deng, C. S. Xia, C. Yang, S. Zhang, S. Yang, L. Zhang, Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 70:1–70:13. doi:10.1145/3597503.3623343.
- [26] P. Srivastava, M. Payer, Gramatron: effective grammar-aware fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 244–256. doi:10.1145/3460319.3464814.
- [27] J. Fu, J. Liang, Z. Wu, M. Wang, Y. Jiang, Griffin: Grammar-free dbms fuzzing, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12. doi:10.1145/3551349.3560431.
- [28] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, Y. Xiang, Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 337–350. doi:10.1145/3460120.3484543.
- [29] C. Salls, C. Jindal, J. Corina, C. Kruegel, G. Vigna, Token-level fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2795–2809.
- [30] A. Bulekov, B. Das, S. Hajnoczi, M. Egele, No grammar, no problem: Towards fuzzing the Linux kernel without system-call descriptions, in: 30th Annual Network and Distributed System Security Symposium (NDSS 2023), 2023, pp. 1–16. doi:10.14722/ndss.2023.24688.
- [31] A. Fioraldi, D. C. D'Elia, D. Balzarotti, The use of likely invariants as feedback for fuzzers, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2829–2846.
- [32] C. Myung, G. Lee, B. Lee, MundoFuzz: Hypervisor fuzzing with statistical coverage testing and grammar inference, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 1257–1274.
- [33] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, C. Zhang, NSFuzz: Towards efficient and state-aware network service fuzzing, ACM Transactions on Software Engineering and Methodology 32 (2023) 160:1–160:26. doi:10.1145/3580598.
- [34] Y. Zhang, D. Fang, P. Liu, L. Xi, X. Lu, X. Chen, S. Si, L. Sun, MSGFuzzer: Message sequence guided industrial robot protocol fuzzing, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), 2024, pp. 140–150. doi:10.1109/icst60714.2024.00021.
- [35] K. Kim, T. Kim, E. Warraich, B. Lee, K. R. B. Butler, A. Bianchi, D. J. Tian, FuzzUSB: Hybrid stateful fuzzing of USB gadget stacks, in: 2022 IEEE Symposium on Security and Privacy (SP), 2022, pp. 2212–2229. doi:10.1109/sp46214.2022.9833593.
- [36] P. Fiterau-Brosteau, B. Jonsson, K. Sagonas, F. Taquist, DTLs-Fuzzer: A DTLs protocol state fuzzer, in: 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), 2022, pp. 456–458. doi:10.1109/icst53961.2022.00051.
- [37] K. Sagonas, T. Typaldos, EDHOC-Fuzzer: An EDHOC protocol state fuzzer, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 1495–1498. doi:10.1145/3597926.3604922.
- [38] Q. Zhang, Y. Chu, Y. Shen, J. Liu, H. Shi, Y. Jiang, W. Chang, Tron: Fuzzing Linux network stack via protocol-system call payload synthesis, in: 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025, pp. 3227–3238. doi:10.1109/ase63991.2025.00266.
- [39] H. Kim, M. O. Ozmen, A. Bianchi, Z. B. Celik, D. Xu, PGFUZZ: Policy-guided fuzzing for robotic vehicles, in: 28th Annual Network and Distributed System Security Symposium (NDSS 2021), 2021, pp. 1–18. doi:10.14722/ndss.2021.24096.
- [40] Y. Aafer, W. You, Y. Sun, Y. Shi, X. Zhang, H. Yin, Android

- smarttvs vulnerability discovery via log-guided fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2759–2776.
- [41] T. Scharnowski, N. Bars, M. Schloegel, E. Gustafson, M. Muench, G. Vigna, C. Kruegel, T. Holz, A. Abbasi, Fuzzware: Using precise MMIO modeling for effective firmware fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 1239–1256.
- [42] B. Zhao, Z. Li, S. Qin, Z. Ma, M. Yuan, W. Zhu, Z. Tian, C. Zhang, StateFuzz: System call-based state-aware linux driver fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3273–3289.
- [43] D. Ryu, G. Kim, D. Lee, S. Kim, S. Bae, J. Rhee, T. Kim, Differential fuzzing for data distribution service programs with dynamic configuration, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 807–818. doi:10.1145/3691620.3695073.
- [44] T. Trippel, K. G. Shin, A. Chernyakhovsky, G. Kelly, D. Rizzo, M. Hicks, Fuzzing hardware like software, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3237–3254.
- [45] M. Beckmann, J. Steffan, Coverage-guided fuzzing of embedded systems leveraging hardware tracing, in: Computer Security – ESORICS 2023, 2023, pp. 362–378. doi:10.1007/978-3-031-25460-4_21.
- [46] Z. Du, Y. Li, Y. Liu, B. Mao, WindRanger: A directed greybox fuzzer driven by deviation basic blocks, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2440–2451. doi:10.1145/3510003.3510197.
- [47] P. Lin, P. Wang, X. Zhou, W. Xie, X. Ren, K. Lu, When control flows deviate: Directed grey-box fuzzing with probabilistic reachability analysis, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering, 2025, pp. 713–725. doi:10.1109/ASE63991.2025.00065.
- [48] Y. Chen, R. Zhong, H. Hu, H. Zhang, Y. Yang, D. Wu, W. Lee, One Engine to Fuzz 'em All: Generic language processor testing with semantic validation, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 642–658. doi:10.1109/SP40001.2021.00071.
- [49] C. Zhang, X. Lin, Y. Li, Y. Xue, J. Xie, H. Chen, X. Ying, J. Wang, Y. Liu, APICraft: Fuzz driver generation for closed-source sdk libraries, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2811–2828.
- [50] J. Jiang, H. Xu, Y. Zhou, RULF: Rust library fuzzing via api dependency graph traversal, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 581–592. doi:10.1109/ASE51524.2021.9678813.
- [51] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, S. K. Cha, SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 227–239. doi:10.1109/ASE51524.2021.9678888.
- [52] Z. Zhou, Y. Yang, S. Wu, Y. Huang, B. Chen, X. Peng, Magneto: A step-wise approach to exploit vulnerabilities in dependent libraries via llm-empowered directed fuzzing, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1633–1644. doi:10.1145/3691620.3695531.
- [53] D. Xie, Y. Li, M. Kim, H. V. Pham, L. Tan, X. Zhang, M. W. Godfrey, DocTer: documentation-guided fuzzing for testing deep learning API functions, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 176–188. doi:10.1145/3533767.3534220.
- [54] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, T. Holz, SoK: Prudent evaluation practices for fuzzing, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1974–1993. doi:10.1109/sp54263.2024.00137.
- [55] M. Böhme, L. Szekeres, J. Metzmann, On the reliability of coverage-based fuzzer benchmarking, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1621–1633. doi:10.1145/3510003.3510230.
- [56] C. Poncelet, K. K. Sagonas, N. Tsiftes, So many fuzzers, so little time: Experience from evaluating fuzzers on the contiki-ng network stack, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13. doi:10.1145/3551349.3556946.